

FACULTY OF ENGINEERING
Department of Mechatronics Engineering



IoT & Data Analytics Course Project
Smart Home Appliance Energy Consumption Prediction

Group Members:	Student Numbers:
Nora Yahya Ahmed Al-dheeb	231023204
Zhala Mammadli	231023211
Razaz Salih Mohamedali Omer	231023204

Prof. Mehmet Ali Akyol

Table of Contents

Abstract	1
Introduction	1
1. Development Environment (Google Colab)	2
2. Dataset Loading and Initial Inspection	2
3. Sensor Description and Feature Definitions	2
4. Data Cleaning and Preprocessing	4
5. Exploratory Data Analysis (EDA)	4
Figure 1. Correlation heatmap of the selected features used in the energy consumption dataset	5
6. Data Preprocessing and Feature Engineering.....	5
6.1 Date-Time Processing.....	5
6.2 Feature Selection	6
7. Feature Scaling	6
8. Model Training and Evaluation	6
8.1 Train-Test Split	6
8.2 Models Evaluated	6
8.3 Model Selection	7
8.4 Model Interpretation	7
9. Local Dashboard Development (Streamlit).....	7
10. Local Execution of the Dashboard	7
Figure 2. Streamlit-based dashboard interface for appliance energy consumption prediction. .	8
Figure 3. Dashboard interface with example sensor input values entered for selected indoor variables.	8
10.1 System Workflow	9
11. Validation and Testing	9
12. Dashboard Deployment Using AWS EC2 and PowerShell	9
12.1 Purpose of the Dashboard	9
13. Tools and Technologies Used	10
14. AWS EC2 Setup	10
14.1 Creating the EC2 Instance	10
Figure4 : AWS EC2 instance used for the project, shown in the Running state in the AWS Management Console	11

Figur5 : Details page of the AWS EC2 instance used in the project	11
15. Connecting to AWS Using PowerShell	11
15.1 Preparing the SSH Key	12
15.2 Connecting to the EC2 Instance	12
16. Server Environment Setup on AWS	12
16.1 System Update	12
16.2 Creating and Activating a Virtual Environment	12
16.3 Installing Required Python Libraries	13
17. Running and Displaying the Dashboard	13
Figure 6: The deployed Streamlit dashboard displayed in Google Chrome after successfully running the application on the AWS EC2 instance.....	14
Figure 7 : Dashboard interface showing user input values set to 0.01 for all sensor parameters and the resulting energy consumption prediction displayed on the screen.	14
Conclusion.....	15

Abstract

This project presents the design and implementation of a smart energy consumption prediction system for residential buildings using machine learning techniques. The system utilizes environmental and temporal sensor data collected from a smart home to predict household appliance energy consumption. Data preprocessing, feature engineering, and model development were performed using Python in a Google Colab environment. Several regression models were evaluated, including Linear Regression, Decision Tree, Random Forest, and Gradient Boosting, with the Random Forest Regressor achieving the best predictive performance.

To demonstrate practical usability, the trained model was integrated into an interactive Streamlit dashboard that allows real-time energy consumption prediction based on user inputs. Finally, the dashboard was deployed on an Amazon Web Services (AWS) EC2 instance, enabling remote access through a web browser. The project highlights the integration of data analytics, machine learning, and cloud computing to provide an accessible and scalable energy prediction solution.

Introduction

With the increasing demand for energy efficiency and smart home technologies, predicting household energy consumption has become an important research and practical challenge. Accurate energy consumption forecasting enables better energy management, cost reduction, and informed decision-making for both users and utility providers.

This project focuses on developing a machine learning-based system capable of predicting appliance energy consumption using data collected from multiple indoor environmental sensors and external weather measurements. The dataset includes temperature, humidity, and weather-related variables, as well as time-based features that capture daily and seasonal usage patterns.

The project follows a complete end-to-end workflow, starting from data loading and preprocessing, through exploratory data analysis and model training, and ending with real-world deployment. An interactive dashboard was developed using Streamlit to make the predictive model accessible to non-technical users. To further enhance practicality, the dashboard was deployed on a cloud server using AWS EC2, allowing users to access the system remotely.

The main objective of this project is to demonstrate how machine learning models can be combined with web-based interfaces and cloud infrastructure to build an intelligent, scalable, and user-friendly energy prediction system.

Data Processing, Model Development, and Local Deployment (Google Colab & Python)

1. Development Environment (Google Colab)

Google Colab was used as the primary environment for data preprocessing, analysis, and machine learning model development. Colab was chosen because it provides a cloud-based Python environment with pre-installed data science libraries, allowing fast experimentation without local setup constraints.

The dataset (energydata_complete.csv) was uploaded directly into the Colab environment and loaded using the Pandas library for structured data handling.

2. Dataset Loading and Initial Inspection

The dataset was loaded using Pandas, ensuring the correct delimiter was applied. An initial inspection was performed to understand the structure of the data, including column names, data types, and the presence of missing or unnecessary variables.

Basic checks included:

- Viewing the first and last rows of the dataset
- Checking the dataset shape
- Verifying column data types
- Confirming there were no missing values requiring imputation
- This step ensured that the dataset was complete and suitable for further processing.

3. Sensor Description and Feature Definitions

The original dataset contains several abbreviated sensor names corresponding to temperature and humidity measurements collected from different locations inside the smart home. To ensure clarity and interpretability, the sensor abbreviations used in the dataset are explained below.

Sensor Name	Description
Appliances	Total energy consumption of household appliances measured in watt-hours (Wh). This is the target variable used for prediction.
lights	Energy consumption of lighting fixtures measured in watt-hours (Wh).
T1	Temperature in the kitchen (°C).
RH_1	Relative humidity in the kitchen (%).
T2	Temperature in the living room (°C).
RH_2	Relative humidity in the living room (%).

Sensor Name	Description
T3	Temperature in the laundry room (°C).
RH_3	Relative humidity in the laundry room (%).
T4	Temperature in the office room (°C).
RH_4	Relative humidity in the office room (%).
T5	Temperature in the bathroom (°C).
RH_5	Relative humidity in the bathroom (%).
T6	Temperature outside the building (north side) (°C).
RH_6	Relative humidity outside the building (%).
T7	Temperature in the ironing room (°C).
RH_7	Relative humidity in the ironing room (%).
T8	Temperature in the teenager room (°C).
RH_8	Relative humidity in the teenager room (%).
T9	Temperature in the parent room (°C).
RH_9	Relative humidity in the parent room (%).
T_out	Outdoor temperature measured at the weather station (°C).
RH_out	Outdoor relative humidity (%).
Press_mm_hg	Atmospheric pressure measured in millimeters of mercury (mmHg).
Windspeed	Wind speed measured in meters per second (m/s).
Visibility	Outdoor visibility measured in kilometers (km).
Tdewpoint	Outdoor dew point temperature (°C).
hour	Hour of the day extracted from the timestamp.
day_of_week	Day of the week extracted from the timestamp (0 = Monday, 6 = Sunday).
month	Month of the year extracted from the timestamp.

Sensor Name	Description
week_of_year	Week number of the year extracted from the timestamp.

This section ensures that all sensor variables used in the analysis are clearly defined, improving the readability and transparency of the modeling process.

4.Data Cleaning and Preprocessing

The dataset was first loaded using the Pandas library. Since the original data file uses a semicolon (;) as a separator, the appropriate delimiter was specified during loading to ensure correct parsing of columns.

The date column was converted into a datetime format to enable time-based feature extraction. From this timestamp, additional temporal features were engineered, including hour of the day, day of the week, month, and week of the year. These features were added to capture daily and seasonal energy consumption patterns.

Columns not used for prediction, including the original date column, were removed. The target variable was defined as the appliance energy consumption (Appliances), while all remaining columns were treated as input features.

To ensure consistent scaling across features with different units and ranges, the dataset was standardized using StandardScaler. This step is essential for improving model stability and performance, especially for distance-based and ensemble models.

5. Exploratory Data Analysis (EDA)

Before training machine learning models, an exploratory data analysis was conducted to understand the relationships between variables and appliance energy consumption.

Correlation analysis was performed using a heatmap to visualize dependencies between temperature, humidity, weather variables, and energy usage. This step helped identify meaningful patterns and confirmed that indoor environmental conditions and time-related features influence appliance energy consumption.

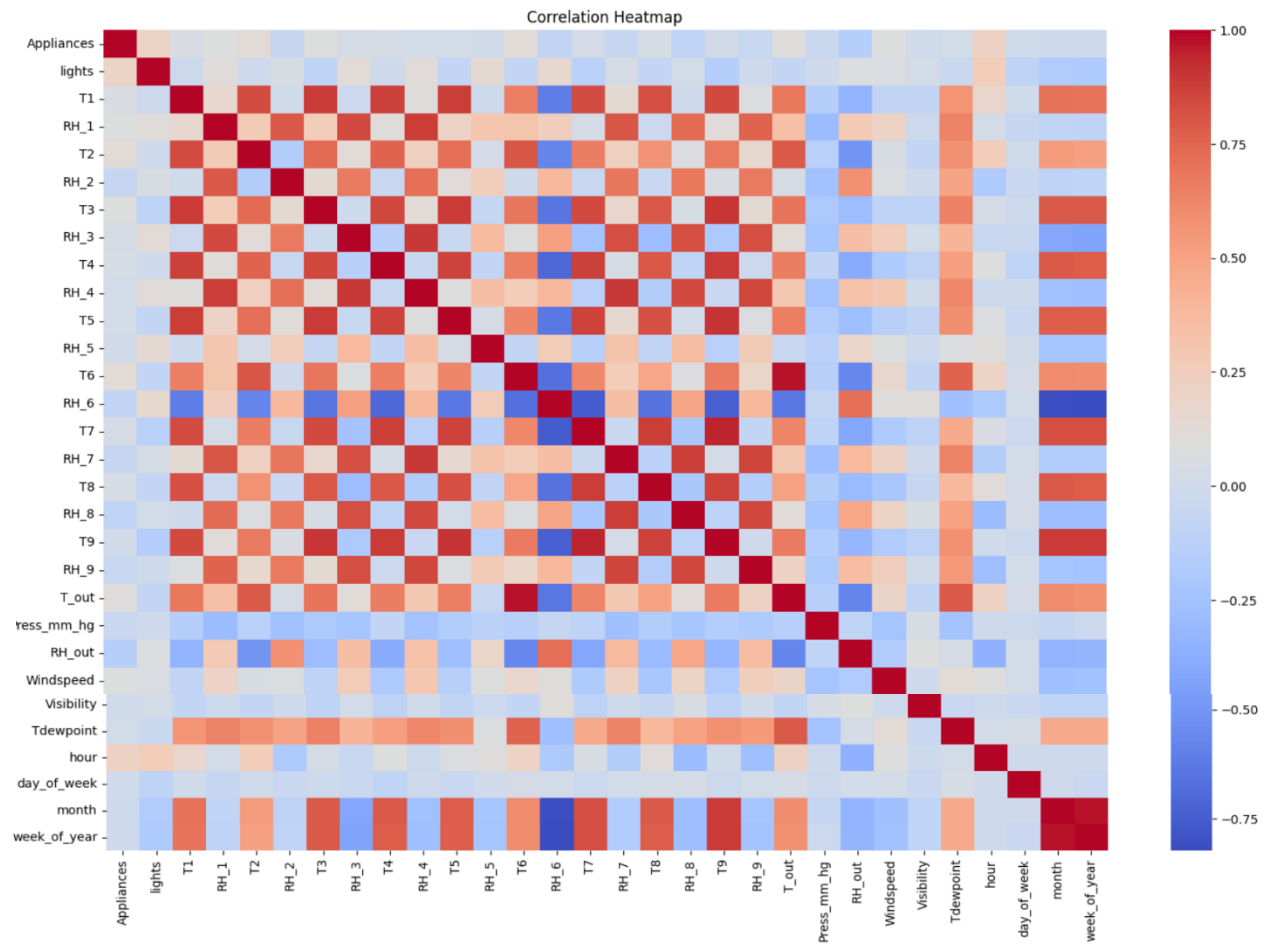


Figure 1. Correlation heatmap of the selected features used in the energy consumption dataset.

The heatmap illustrates the pairwise correlations between input variables and the target energy consumption. This analysis was used during the exploratory data analysis phase to better understand relationships among features and to identify potential dependencies or redundancies before model training.

6. Data Preprocessing and Feature Engineering

6.1 Date-Time Processing

The date column was converted into a proper datetime format. From this timestamp, additional time-based features were extracted to capture daily and seasonal energy usage patterns:

- Hour of the day
- Day of the week
- Month
- Week of the year

These features help the model learn how energy consumption varies across time.

6.2 Feature Selection

The following columns were removed:

date (already transformed into numerical features)

rv1 and rv2 (random variables included only for regression testing and not meaningful for prediction)

All remaining columns were retained as predictive features.

7. Feature Scaling

Before model training, the feature set was standardized using StandardScaler.

Scaling was necessary because the dataset contains features with different units and ranges (e.g., temperature, humidity, pressure, wind speed).

Standardization ensures:

- Faster model convergence
- Fair contribution of all features
- Improved performance for tree-based ensemble models

The fitted scaler was saved (scaler.pkl) for later use in the dashboard to ensure consistent preprocessing of user inputs.

8. Model Training and Evaluation

8.1 Train-Test Split

Multiple regression models were trained and evaluated to identify the most suitable approach for predicting appliance energy consumption. The models included Linear Regression, Decision Tree Regression, Random Forest Regression, and Gradient Boosting Regression.

The dataset was split into training and testing subsets using an 80/20 ratio. Models were trained on the training set and evaluated on unseen test data to assess generalization performance.

The dataset was split into:

- 80% training data
- 20% testing data

This allowed objective evaluation of model performance on unseen data.

8.2 Models Evaluated

Several regression models were trained and compared:

- Linear Regression
- Decision Tree Regressor
- Random Forest Regressor

- Gradient Boosting Regressor

Each model was evaluated using:

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)

8.3 Model Selection

Among all models tested, the Random Forest Regressor achieved the best balance between accuracy and stability, producing the lowest prediction error.

This model was selected as the final predictive model and saved as:

random_forest_model.pkl

Although prediction errors were not zero, the achieved MAE values are reasonable given the stochastic nature of human behavior and appliance usage. Energy consumption depends on factors not fully captured by sensors, such as occupant activity, making perfect prediction unrealistic.

8.4 Model Interpretation

To better understand the model's behavior, feature importance scores from the Random Forest model were examined. The most influential features included indoor temperature and humidity sensors as well as time-based variables such as hour of the day.

This confirms that appliance energy consumption is strongly linked to both environmental conditions and daily usage patterns.

9. Local Dashboard Development (Streamlit)

To demonstrate the practical application of the trained model, an interactive dashboard was developed using Streamlit.

The dashboard:

- Loads the trained Random Forest model and the saved scaler
- Provides input fields for all sensor values
- Applies the same preprocessing steps used during training
- Predicts appliance energy consumption in real time
- Displays the predicted value in watt-hours (Wh)

This allows users to interact with the model without needing programming knowledge.

10. Local Execution of the Dashboard

The dashboard was tested locally before cloud deployment.

Steps followed:

- All required Python libraries were installed using requirements.txt
- The Streamlit application was launched using: streamlit run app.py

The dashboard opened in a web browser

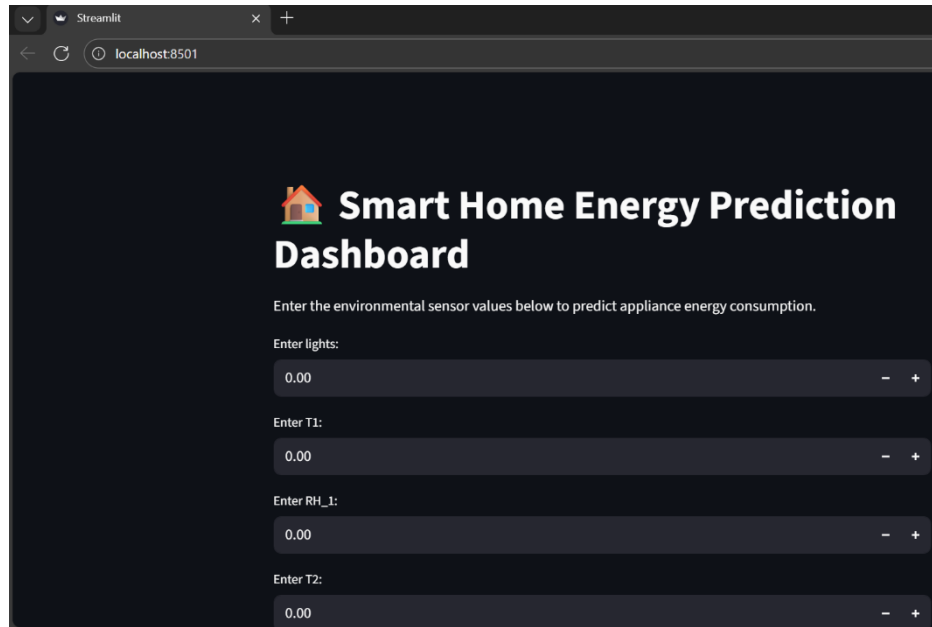


Figure 2. Streamlit-based dashboard interface for appliance energy consumption prediction.

The figure presents the user interface of the Streamlit dashboard developed for this project. The dashboard provides structured input fields corresponding to indoor sensor measurements and weather related variables. Once values are entered, the system preprocesses the inputs and generates an appliance energy consumption prediction using the trained Random Forest model.

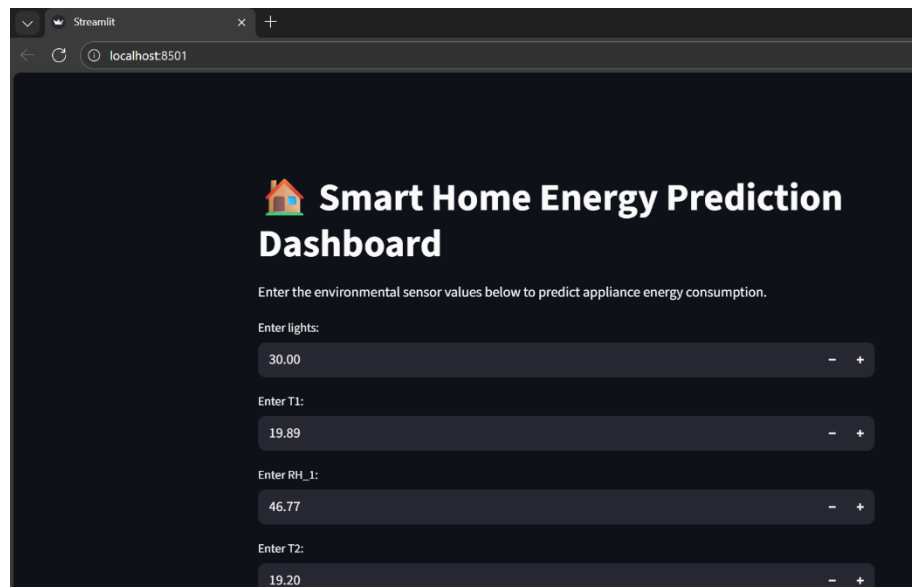


Figure 3. Dashboard interface with example sensor input values entered for selected indoor variables.

Sample inputs from the dataset were manually entered to validate predictions

To demonstrate user interaction, a second interface view is shown with representative numerical values entered for a subset of sensor inputs. The prediction output is intentionally omitted to keep the focus on data entry flow and interface functionality rather than specific numerical results, which are evaluated separately using quantitative performance metrics.

This confirmed that:

- The model loads correctly
- Inputs are processed properly
- Predictions respond dynamically to changes in sensor values

10.1 System Workflow

The overall system workflow consists of four main stages: data collection, preprocessing, model training, and prediction deployment. Sensor data is first cleaned and processed, then used to train a machine learning model. The trained model is integrated into a Streamlit dashboard, allowing real-time energy consumption prediction based on user inputs.

11. Validation and Testing

To ensure correct behavior, real rows from the dataset were used as test inputs in the dashboard. The predicted values were compared with actual appliance energy values to verify that predictions were reasonable and responsive to feature changes.

Minor deviations were expected due to the probabilistic nature of ensemble models, but overall predictions followed realistic energy consumption trend

12. Dashboard Deployment Using AWS EC2 and PowerShell

This part provides a comprehensive explanation of the process used to deploy an interactive **machine learning dashboard** on the cloud using **Amazon Web Services (AWS)**. The dashboard was designed to allow users to input numerical data and receive predictive results in real time. The deployment was carried out using an **AWS EC2 instance** running **Ubuntu Linux**, while remote access and management were performed using **Windows PowerShell** through the **SSH protocol**.

The project combines cloud computing, web-based dashboards, and machine learning into a single practical application that demonstrates how theoretical concepts can be applied in a real-world environment.

12.1 Purpose of the Dashboard

The dashboard is a **machine learning–based prediction system**. It provides a graphical user interface (GUI) that allows users to enter multiple numerical features related to a specific system (for example: environmental measurements, sensor readings, or operational parameters).

Based on the user inputs, the dashboard:

- Preprocesses the data using a trained **scaler**
- Sends the processed data to a trained **machine learning model**
- Displays a predicted output instantly on the screen

The dashboard removes the need for users to interact directly with code, making advanced machine learning models accessible through a simple web interface.

13. Tools and Technologies Used

Technology	Description
AWS EC2	Cloud-based virtual server
Ubuntu Linux	Operating system on the server
Windows PowerShell	Remote server access using SSH
Python 3	Programming language
Virtual Environment (venv)	Python dependency isolation
Streamlit	Framework for building dashboards
Scikit-learn	Machine learning model library
SSH Key (.pem)	Secure authentication

14. AWS EC2 Setup

14.1 Creating the EC2 Instance

An EC2 instance was created using the AWS Management Console with the following configuration:

- **AMI:** Ubuntu (64-bit)
- **Instance Type:** t2.micro
- **Key Pair:** A private key file (`key.pem`) for SSH authentication
- **Security Group Rules:**
 - Port 22 (SSH) – for remote access
 - Port 8501 – for accessing the Streamlit dashboard

After launching the instance, AWS assigned a **public IPv4 address**, which was later used to access the dashboard from a web browser.

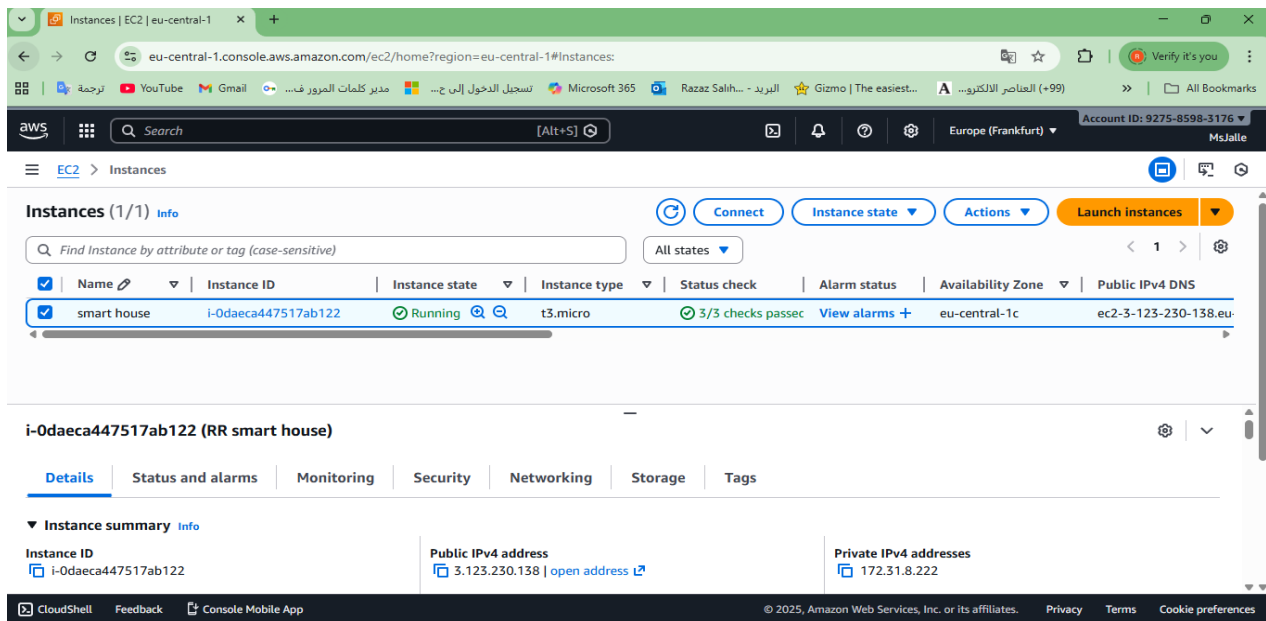
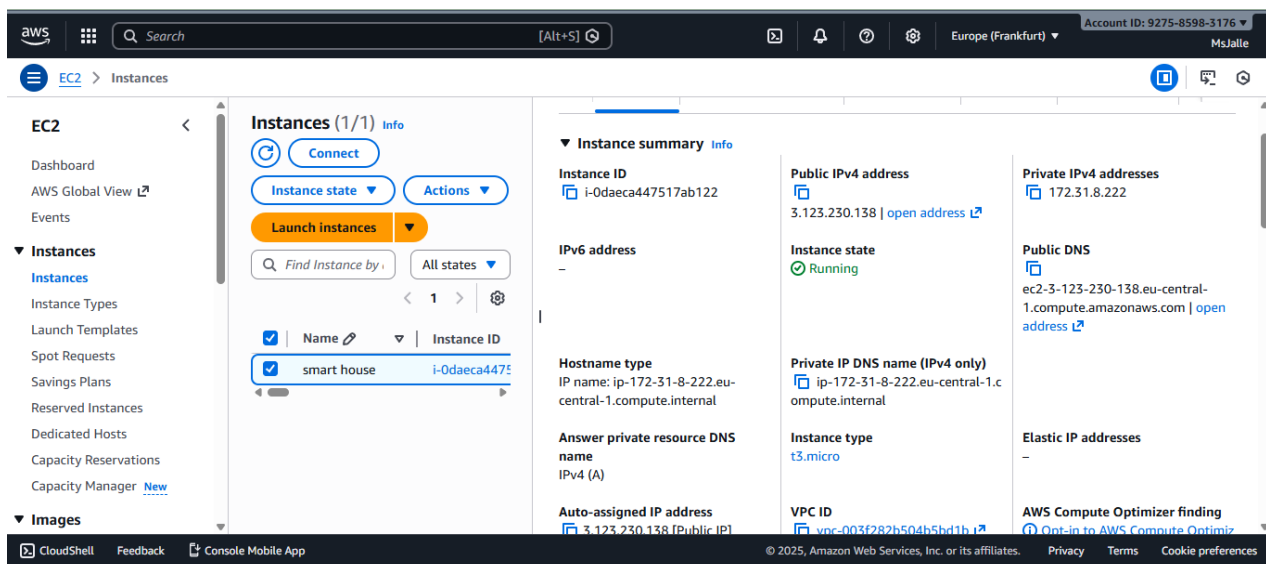


Figure 4 : AWS EC2 instance used for the project, shown in the Running state in the AWS Management Console.



Figur 5: Details page of the AWS EC2 instance used in the project

15. Connecting to AWS Using PowerShell

After creating the EC2 instance, the next step was to establish a secure connection between the local computer (Windows) and the AWS server. This was done using **Windows PowerShell** and the **SSH protocol**.

15.1 Preparing the SSH Key

When the EC2 instance was created, AWS generated a private key file used for authentication. This file was saved locally and renamed to:

```
key.pem
```

The file was stored on the Windows Desktop to simplify access. File permissions were configured to ensure that only the owner could use the key for authentication.

15.2 Connecting to the EC2 Instance

Using PowerShell, the following commands were executed:

```
cd Desktop  
ssh -i key.pem ubuntu@3.123.230.138
```

Here:

- `key.pem` is the SSH private key
- `ubuntu` is the default user for Ubuntu EC2 instances
- `3.123.230.138` is the public IP address assigned by AWS

Once the connection was successful, the terminal displayed the Ubuntu shell prompt, confirming remote access to the server.

16. Server Environment Setup on AWS

16.1 System Update

Before installing any software, the Ubuntu system was updated to ensure security and stability:

```
sudo apt update && sudo apt upgrade -y
```

16.2 Creating and Activating a Virtual Environment

To isolate project dependencies, a Python virtual environment was created:

```
python3 -m venv venv  
source venv/bin/activate
```

After activation, the terminal showed:

```
(venv) ubuntu@ip-172-31-8-222
```

This confirmed that all Python packages would be installed inside the virtual environment.

16.3 Installing Required Python Libraries

The following libraries were installed:

```
pip install streamlit pandas numpy scikit-learn
```

These libraries are essential for data handling, machine learning, and dashboard development.

17. Running and Displaying the Dashboard

After completing the server environment setup and installing all required Python libraries, the final step was to **run the dashboard application and display it through a web browser**.

Once the virtual environment was activated and all dependencies were successfully installed, the Streamlit dashboard was executed on the AWS EC2 instance using the following command:

```
streamlit run app.py --server.port 8501 --server.address 0.0.0.0
```

This command starts the Streamlit application using the Python file `app.py`. The specified port (**8501**) is used to host the dashboard, while the address `0.0.0.0` allows the application to accept incoming connections from external devices.

After running the command, the dashboard service started successfully on the server. The application was then accessed from a local machine using a web browser by entering the public IP address of the EC2 instance followed by the port number:

```
http://3.123.230.138:8501
```

The dashboard interface was displayed correctly in Google Chrome, confirming that the application was running properly on the cloud server and that remote access had been successfully established.

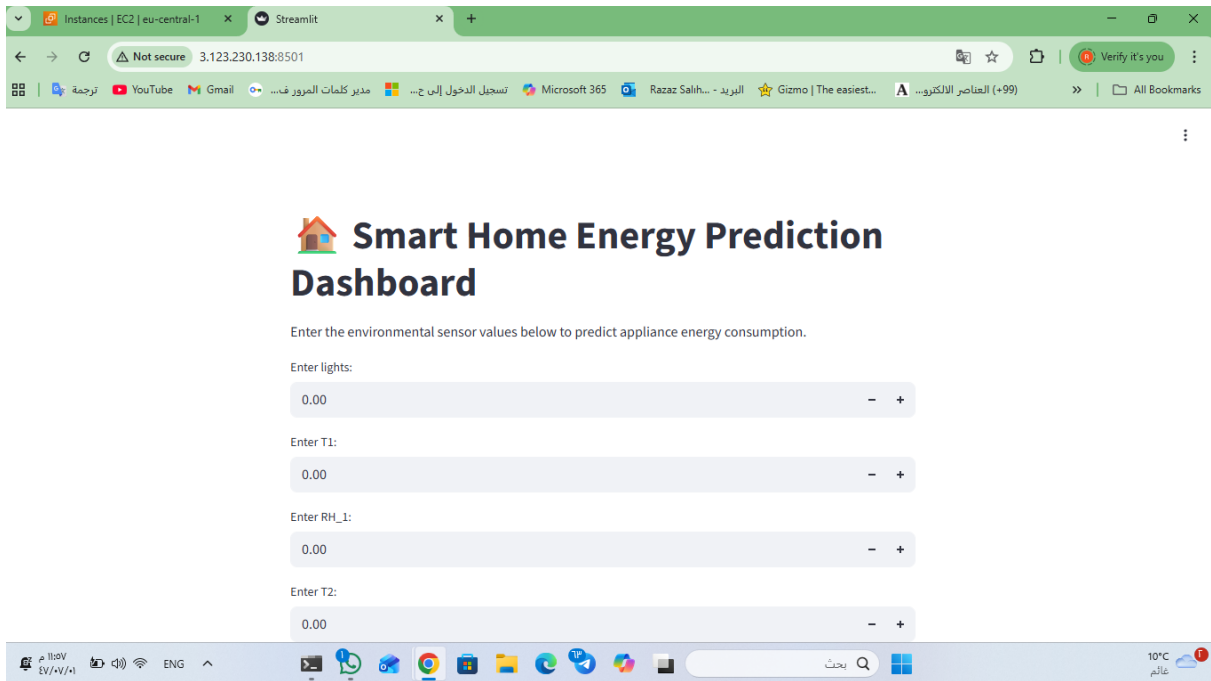


Figure 6: The deployed Streamlit dashboard displayed in Google Chrome after successfully running the application on the AWS EC2 instance.

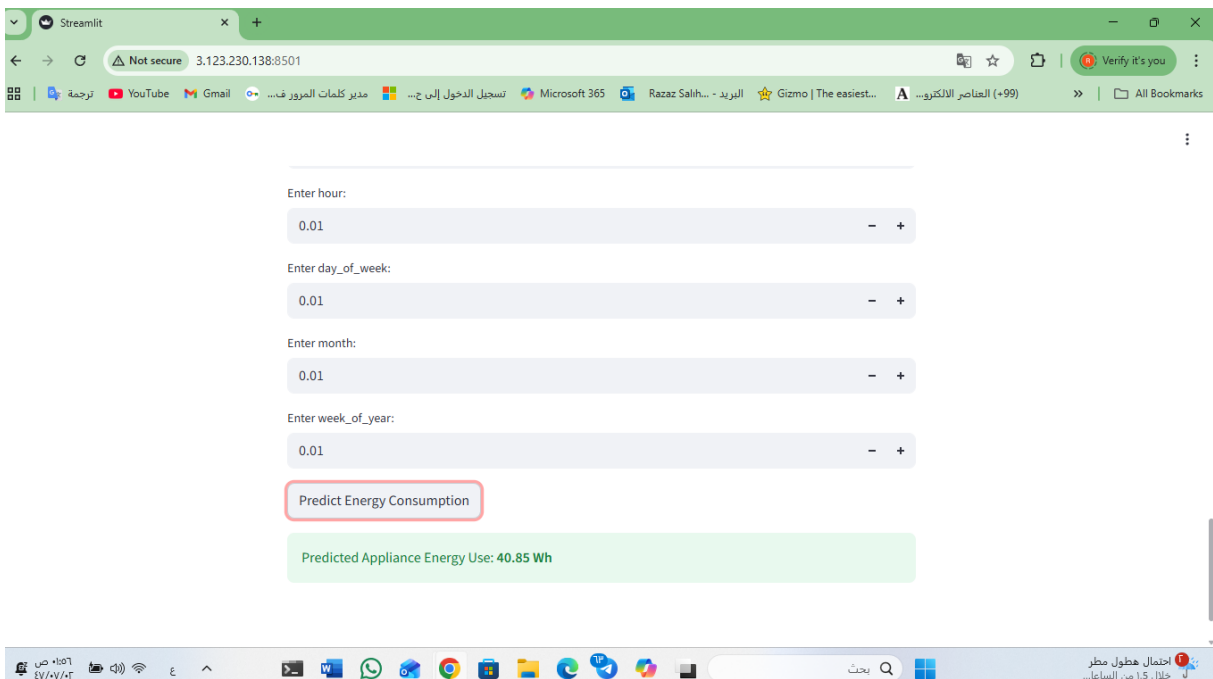


Figure 7 : Dashboard interface showing user input values set to 0.01 for all sensor parameters and the resulting energy consumption prediction displayed on the screen.

Conclusion

This project successfully demonstrates the development and deployment of a machine learning-based energy consumption prediction system. By utilizing sensor data from a smart home environment, meaningful features were extracted and used to train several regression models. Among the evaluated models, the Random Forest Regressor provided the best performance in terms of accuracy and stability.

The integration of the trained model into a Streamlit dashboard transformed the predictive system into an interactive and user-friendly application, allowing real-time predictions without requiring programming knowledge. Furthermore, deploying the dashboard on an AWS EC2 instance showcased how cloud computing can be used to host and scale machine learning applications efficiently.

Overall, the project bridges the gap between theoretical machine learning concepts and real-world applications by combining data analysis, model development, visualization, and cloud deployment. The proposed system can be extended in the future by incorporating additional sensors, advanced models, or real-time data streams to further improve prediction accuracy and system functionality.